

Project V1

Description: This project was started during Week 3 of Term 1, where I have just learnt basic C++ operational and coding skills such as I/O manipulators and operators.

Problem Statement:

Aircraft takeoff performance is affected by multiple environmental and operational factors. Calculating takeoff distance manually can be complex and time-consuming.

This project aims to develop a simplified C++-based calculator that estimates aircraft takeoff distance using user-provided inputs and aviation performance formulas.

Objectives:

- Develop a C++ Program that estimates aircraft takeoff distance
- Apply basic aviation and physics principles in programming
- Allow users to input aircraft and environment specifications
- Determine whether a runway length is sufficient for takeoff
- Improve problem-solving skills

Scope of Project:

It will:

- Allow the user to input aircraft information
- Output estimated takeoff distance and runway safety margin
- Estimate takeoff distance using simplified calculations

It will NOT:

- Include real-time weather data
- Simulate full aerodynamic physics
- Handle emergency procedures or engine failures

Features:

The program shall:

1. Allow the user to input aircraft inputs
2. Accept aircraft weight input
3. Accept aircraft thrust input
4. Accept aircraft wing area input
5. Accept aircraft wing height input
6. Accept aircraft $C_{L,max}$ input
7. Accept aircraft C_{D0} input
8. Accept aircraft $C_{D,config}$ input
9. Accept runway length input
10. Accept runway weather condition input
11. Accept runway altitude input
12. Accept runway temperature input
13. Accept runway humidity input
14. Calculate estimated takeoff distance
15. Compare required distance with available runway
16. Display the safety of the takeoff and required runway distance

Assumptions:

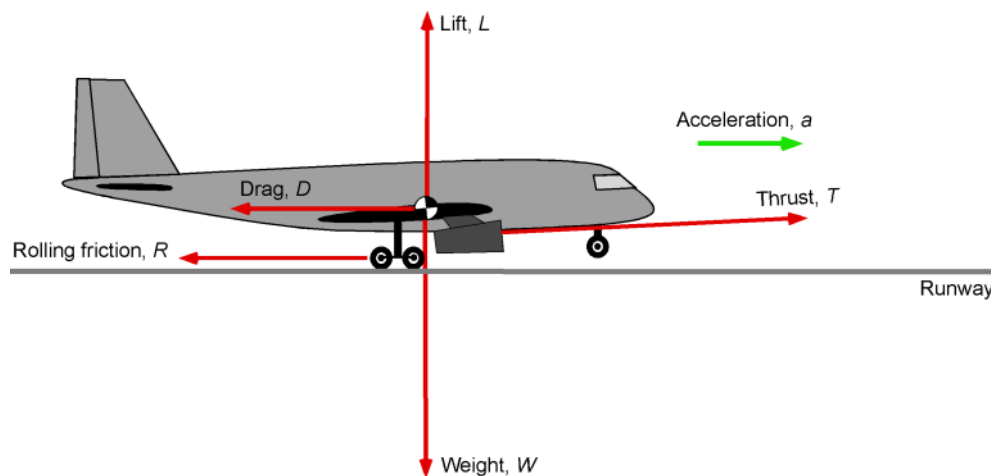
The following assumptions are adopted in the development of this program:

1. The runway is assumed to be level, paved, and in average condition throughout the takeoff roll.
2. The thrust produced by the engines is assumed to act parallel to the aircraft's direction of motion. Any thrust inclination angle is considered negligible.
3. Aircraft weight is assumed to remain constant during the takeoff roll. The effects of fuel consumption are neglected due to the relatively short duration of the takeoff phase.
4. The aircraft is assumed to remain in a constant takeoff configuration, with no changes to flap settings, slat positions, or landing gear configuration during the simulation.

5. The engines are assumed to provide a constant takeoff thrust from the start of the takeoff roll ($t = 0$). Engine spool-up characteristics are not considered.
6. All aircraft systems and engines are assumed to operate normally, with no mechanical failures or performance degradation.
7. The runway is assumed to be free of obstacles and external hazards that may affect the takeoff performance.
8. Environmental and weather conditions, including temperature, air density, and wind conditions, are assumed to remain constant throughout the takeoff roll.
9. Simplified aerodynamic and performance equations are employed to estimate takeoff performance. Consequently, the calculated takeoff distance represents an approximation and may differ from actual aircraft performance.

Definition of takeoff distance: For this program, we will define takeoff distance as the total horizontal distance required for an aircraft to climb from a standstill and lift off the runway ONLY.

Research and Planning



Forces acting on an aircraft during takeoff roll.

The aircraft accelerates because of thrust produced from the engines BUT friction between runways and wheels, and drag from the fuselage and wings of the airplane resists motion. So, Net Force = Thrust - Resistance.

I have chosen to calculate stall speed V_s , the liftoff speed V_{LO} according to V_s , and stop the calculation once $V=V_{LO}$. I have chosen this over stopping the calculation when $L = W$ because in reality, pilots may rotate even before the aircraft is fully airborne, causing a sudden increase in lift generation as the angle of attack increases, which would make the calculation less accurate.

A total runway takeoff safety buffer of 20% is factored in. This means that the aircraft should be airborne by the time it reaches 80% of the runway length, or the takeoff safety status will be displayed as “Unsafe”.

Takeoff Lift equation: $L = \frac{1}{2}\rho V^2 S C_{L,TO}$

where ρ = Air density (assumed to be constant throughout the takeoff roll), V = True Airspeed (how fast the aircraft is moving relative to the air), S = Surface Area of Wing, and C_L = coefficient of lift for takeoff. This proves that there is a positive linear correlation between the lift L and i) the surface area of wing S , ii) the coefficient of lift C_L , and iii) the air density ρ . There is an exponential correlation between the lift L and the airspeed V (ie L is increased by a factor of 4 when V is increased by a factor of 2).

Coefficient of Lift during Takeoff: $C_{L,TO} = 0.8C_{L,max}$

This formula will be assumed for all aircrafts.

Edit: This formula will be changed during the troubleshooting phase.

Takeoff Drag equation: $D = \frac{1}{2}\rho V^2 S C_D = \frac{1}{2}\rho V^2 S (C_{D0} + \phi \frac{C_{LTO}^2}{\pi A R e} + \Delta C_{D,config})$

Similar to Lift Equation, ρ = Air density (assumed to be constant in this project), V = True Airspeed (how fast the aircraft is moving relative to the air), S = Surface Area of Wing, but now, C_D = total cruise coefficient of drag. C_D is a dimensionless number used in physics and engineering to quantify an object's air or fluid resistance. This proves that there is a positive linear correlation between the lift L and 1) the surface area of wing S , 2) the coefficient of drag C_D , and 3) the air density ρ . There is an exponential correlation between the lift L and the airspeed V (ie L is increased by a factor of 4 when V is increased by a factor of 2).

C_{D0} is the zero-lift/parasite drag coefficient (that is, drag NOT caused by lift) and includes skin friction, fuselage drag, landing gear drag, antenna drag, and engine nacelle drag.

$\phi \frac{C_L^2}{\pi A R e}$ is the induced drag, ϕ is the ground effect, C_{LTO} is the takeoff coefficient (increases by a factor of 4), AR is the Aspect Ratio, which measures the square of the wingspan relative to its wing area. Higher AR , less induced drag, more efficient wing.

$AR = \frac{b^2}{s}$ where b is the wingspan and s is the wing area.

e is the Oswald Efficiency Factor, which accounts for non-ideal lift distribution, wingtip effects, and aerodynamics efficiency. The Oswald Efficiency Factor is typically between 0.6-0.9.

$\Delta C_{D,config}$ is the configuration drag. This is the calculated wind resistance added by extending the takeoff flaps and deploying the landing gear.

Coefficient of drag: $C_D = C_{D0} + \phi \frac{C_{LTO}^2}{\pi A R e} + \Delta C_{D,config}$

Ground effect: $\phi = \frac{16(h/b)^2}{1 + 16(h/b)^2}$

where h is the height of the wing above the ground and b is the wingspan. The ground effect is dimensionless and accounts for the reduced induced drag near the runway. Lower ϕ , less induced drag, better acceleration.

Rolling resistance: $R = \mu_r(W - L)$

where R = wheel friction force, μ_r = runway friction coefficient, W = aircraft weight, and L = lift generated. This proves a negative correlation between the rolling resistance R and the lift generated L . At liftoff, $L = W$ and the tyres bear no load, therefore there is zero rolling resistance at liftoff. For a dry, paved runway, the coefficient of friction, μ_r , will be taken to be 0.02. This value will be taken as 0.1 on a wet runway and 0.015 on an icy runway.

Net Force: $F = T - R - D$

where T is the thrust, R is the rolling resistance, and D is the drag.

Stall Speed: $V_s = \sqrt{\frac{2W}{\rho S C_{L,max}}}$

where W is the weight, ρ is the density of air, S is the surface area of the wing, and $C_{L,max}$ is the maximum coefficient of lift.

Absolute Pressure: $P = P_0 \left(1 - \frac{LH}{T_0}\right)^{\frac{GM}{RL}} = 101325 \left(1 - \frac{0.0065H}{288.15}\right)^{5.25588}$

where $P_0 = 101325\text{Pa}$ (Standard sea-level pressure), $T_0 = 288.15\text{K}$ (Standard sea-level temperature), $L = 0.0065\text{K/m}$ (Temperature Lapse Rate), H is the altitude above mean sea level (in metres), and $\frac{GM}{RL} = 5.25588$ (approximate dimensionless constant derived from gravity, air molar mass, the universal gas constant, and the lapse rate).

Absolute Temperature: $T_K = T_c + 273.15$

Saturation Vapour Pressure: $P_{\text{sat}} = 610.7 * 10^{\frac{7.5*TC}{237.3+TC}}$

where T_C = temperature in degrees Celsius.

Actual Vapour Pressure: $P_V = RH * P_{sat}$

where $RH < 1$.

Total Air Density:

Using Ideal Gas Law, $PV=nRT$. $P = \rho RT$, where P is the absolute pressure, ρ is the gas density, R is the specific gas constant and T is the absolute temperature in Kelvin. Our gases in this case will be dry air and water vapour. $R = \frac{Ru}{M}$, where R_u is the universal gas constant and M is the molar mass of the gas. $R_u = 8.314 \text{ J/(molK)}$. For dry air, $M \approx 0.02897 \text{ kg/mol}$. $R_d = \frac{8.314}{0.02897} \approx 287.05 \text{ J/kg}$. For water vapour, $M \approx 0.01802 \text{ kg/mol}$. $R_v = \frac{8.314}{0.01802} \approx 461.495 \text{ J/kg}$.

$$\rho = \frac{Pd}{R_d T} + \frac{Pv}{R_v T} = \frac{P-Pv}{R_d T} + \frac{Pv}{R_v T} = \frac{P-Pv}{287.05 T} + \frac{Pv}{461.495 T}, \text{ where } T \text{ is in Kelvin.}$$

Takeoff Speed: $V_{LO} = 1.2 V_S$

Acceleration: $\alpha = \frac{F}{M} = \frac{dV}{dt}$

Velocity: $V_{new} = V_{old} + \int \alpha dt \approx V_{old} + \alpha dt$, where $\alpha dt = \Delta V$

$\int \alpha$ can be viewed as the area under the acceleration-time graph. We will approximate that area by using tiny rectangles of width $dt = 0.01$, and height as current acceleration. This is also known as Euler's method, where it approximates $V(t) = V_0 + \int_0^t \alpha dt$ as $V_{n+1} = V_n + \alpha_n \Delta t$.

Distance: $s_{n+1} = s_n + \frac{V_n + V_{n+1}}{2} \Delta t = s_{n+1} = s_n + 0.5 (V_n + V_{n+1}) \Delta t$

The velocity was integrated using Euler's method due to its simplicity and computational efficiency. However, the displacement was computed using the trapezoidal rule to improve numerical accuracy by accounting for velocity variation within each time step.

Coding

Pseudocode:

START

Constants $e=0.8$, $\pi=3.14159265$, $g=9.81$, $dt=0.01$

Variable $V=0.0$, $\alpha=0.0$, $s=0.0$, $t=0.0$

User input aircraft total mass (in kg), thrust (in Newtons), wing area (m^2), wing height (m), wing height above ground (m), $C_{L,max}$, C_{D0} , $C_{D,config}$, runway length (m), runway weather condition, runway altitude above mean sea level (m), temperature (in $^{\circ}C$), relative humidity (%), and Oswald Efficiency factor (e).

Ensure mass, thrust, wingspan, wing height above ground, wing area, CL_{max} , runway length is more than 0, $CD0$ is more than or equal to 0, relative humidity is between 0 to 100 inclusive, and runway weather conditions are valid.

Warn the user if temperature is less than $-80^{\circ}C$ or more than $60^{\circ}C$, if mass is more than 1,000,000kg, and if CL_{max} is more than 4.

// Extra steps added to allow user to input derated input and calculate derated thrust during the troubleshooting phase

If the runway weather condition is wet, $\mu_r=0.1$.

If the runway weather condition is dry, $\mu_r=0.02$.

If the runway weather condition is icy, $\mu_r=0.015$.

Calculate weight W ($W=mg$)

Calculate Absolute Temperature T ($T = T_c + 273.15$)

Calculate Absolute Pressure P ($P = 101325(1 - \frac{0.0065H}{288.15})^{5.25588}$)

Calculate Saturation Vapour Pressure P_{sat} ($P_{sat} = 610.7 * 10^{\frac{7.5*TC}{237.3+TC}}$)

Calculate Actual Vapour Pressure P_v ($P_v = \frac{RH}{100} * P_{sat}$)

Calculate air density ρ ($\rho = \frac{P-P_v}{287.05T} + \frac{P_v}{461.495T}$)

Calculate stall speed V_s ($V_s = \sqrt{\frac{2W}{\rho S C_{L,max}}}$)

Calculate $V_{LO} = 1.2V_s$

Calculate $C_{L,TO}$ ($C_{L,TO} = 0.8C_{L,max}$)

Calculate ground effect ϕ ($\phi = \frac{(16(h/b))^2}{1+(16(h/b))^2}$)

Calculate aspect ratio AR ($AR = \frac{b^2}{s}$)

Calculate coefficient of drag C_D ($C_D = C_{D0} + \phi \frac{CLT O^2}{\pi A Re} + \Delta C_{D,config}$)

While $V < V_{LO}$ {

Calculate D ($D = \frac{1}{2}\rho V^2 S C_D$)

Calculate L ($L = \frac{1}{2}\rho V^2 S C_{L,TO}$)

Calculate rolling resistance R ($R = \mu_r(W-L)$)

Calculate net force F ($F = T - R - D$)

// Extra steps added here to account for decreasing actual thrust during the troubleshooting phase

Calculate acceleration α ($\alpha = \frac{F}{M}$)

$V_{old} = V$

Calculate velocity ($V = V + \alpha dt$)

Calculate distance ($s = s + 0.5(V + V_{old}) dt$)

Calculate time ($t = t + dt$) }

If $s < 0.8 * \text{runway length}$, the safety of takeoff is safe.

Output takeoff distance, takeoff time, takeoff speed, and safety of takeoff

C++ Declarations

Constants

PI 3.14159265, G 9.81, dt 0.01

User Inputs

Double: mass, thrust, S, b, h, CLmax, CD0, CDconfig, runwaylength, alt, tempC, rh, e

// Additional inputs for deratedinput added here

String: runwaycondition, takeoff safety

Environment Variables

Double: tempK, P, Psat, Pv, rho, mew

Aircraft Variables

Double: CLto, AR, phi, CD

// Additional inputs for deratedthrust, and thrustActual added here

Performance Variables

Double: weight, V, Vs, Vlo, V_old

String: takeoffsafety

Simulation Variables

Double: t, s, L, D, R, F, α

How do <limits> work?

This is a library that gives information about data types.

`cin.fail()` becomes true when the computer receives an alphabetic input instead of a numeric input, like when “abc” is inputted into a double variable type.

After an invalid input like the one above, the input stream enters an error state. `cin.clear` is used to reset the state and allow the input stream to accept subsequent inputs.

`cin.ignore(..., '\n')` tells the input stream (`cin`) to skip over characters. It will only stop skipping when it hits the maximum number of characters allowed (... part) or it finds a newline character ('\n' part). `(numeric_limits<streamsize>::max())` sets the maximum number of characters to ignore, which in this case is “the largest number possible”. This tells the computer the absolute maximum size of the input memory. Thus, `cin.ignore(numeric_limits<streamsize>::max(), '\n')` clears out the computer’s input memory. It ignores the rest of the current line up to \n (enter). '\n' is the delimiter, the character after which `cin` stops ignoring.

With that, I am able to make functions that allow me to deal with invalid alphabetic and symbolic inputs.

For input in a certain range to be valid,

Use `while(true)` loop, use `if cin.fail()`, `cin.clear()` and `cin.ignore(..., '\n')` to deal with invalid inputs, and `minVal/maxVal` check if the value is in range.

For any other string input other than wet, dry, icy to be invalid.

Use `while(true)` loop, if `cin` condition is wet, dry, or icy, then return condition. Else cout invalid output and let user reenter conditions.

To warn user if temperature is less than -80°C or more than 60°C,

Use `while(true)` loop, use `if cin.fail()`, `cin.clear()` and `cin.ignore(..., '\n')` to deal with invalid inputs, and add `warnMin` and `warnMax` to check if value is in range of warning.

To warn user if mass is more than 1,000,000kg,

Use the first function (checking range) and add in `warnMax` to warn the user if the value is more than 1,000,000kg.

To warn user if `CLmax` is more than 4,

Use the first function and add in `warnMax` to warn the user if the value is more than 4.

Troubleshooting/Improvement

I noticed that by using thrust as the maximum thrust, I produced consistently minimal runway length used. Thus, I allowed the user to input the percentage of total thrust used (such as using the FLX/MCT settings in the actual cockpit) so that I am able to produce more realistic results. I noticed that my rolling resistance R decreased past 0 into the negative region even before V_{LO} is reached. Thus, I added a code where if $R < 0$, $R = 0$. This, however, means that $L > W$ of the aircraft, though it hasn't yet reached V_{LO} .

1. I noticed my values for takeoff were 50-55% of actual realistic values because I assumed $C_{L,TO}$ to be constant throughout, when in reality, C_L starts off at about 0.3-0.5 and slowly increases to $C_{L,TO} = 0.8C_{L,Max}$ at rotation. Since I'm using $C_{L,TO}$ from $V=0$, too much lift is generated early which causes my R to drop faster than in reality. Less rolling resistance thus leads to shorter distance for takeoff.
2. In addition, real thrust drops with airspeed. At V_{LO} , actual thrust could be 10-15% lower than static thrust, meaning less acceleration is available.

Therefore, I tweaked my code. Instead of fixed CL_{to} , I scaled it with velocity where $CL_{to} = 0.8 * CL_{max} * (V / V_{LO})$. This fixed the issue of (1) where my $C_{L,TO}$ is assumed to be constant.

I also added in a code (`double thrustActual = thrust * (1.0 - 0.5 * (V / 340.0));`) where 0.5 is a typical jet thrust lapse constant, and 340.0 is the approximate speed of sound at sea level in m/s.

Final Code: [Runway Takeoff Code](#)

Final Reflection

Overall, when I worked on this project, I learned a lot about how engineering models are made. The initial brainstorming on what project I wanted to work on was a task that required me to compare the benefits of working on each of it, before I eventually settled on a takeoff runway distance calculator where I was able to challenge myself on problem solving, apply the basics of what I have learnt in C++, push me to further my knowledge and pick up new skills to complete my project, and use physics and mathematics systematically to code this program.

At first, the physics equations appeared simple, but when I tried to put them into a simulation, I had to think carefully about the assumptions I made, the units I used and the physical limits. When I was working on it I had some problems like the aircraft behaving in unrealistic ways, having negative velocities, or it taking too long to take off. Rather than just treating them as coding mistakes, I learned to look at the physics behind it and figure out if the problem was because of wrong assumptions, wrong numbers or wrong logic.

One big thing I learned is the importance of checking and validating the model. By comparing my simulation outputs against expected aircraft performance, I was able to identify discrepancies and refine the model. This taught me that engineering is not just about getting an answer, and a correct one, but about making sure whether or not the answer is physically reasonable and supported by evidence or not.

I also gained exposure to methods like Euler's method and the trapezoidal rule, which helped me approximate how the aircraft motion with respect to time. This showed me how computers can be used to model things when exact answers and solutions are impractical.

Beyond technical knowledge, the project strengthened my problem-solving and perseverance. Many challenges required multiple rounds of testing and debugging before a satisfactory solution was achieved. This experience taught me the great value of having a systematic approach, where complex problems are broken down into smaller components that can be analysed individually. I also trained my willpower and determination as this was a self-initiated project which meant I had to actively keep myself on track, plan and balance my school, extra-curriculars and this project.

As someone with a strong interest in aerospace engineering and aviation, this project provided an opportunity for me to apply concepts from aerodynamics, mechanics, and programming in an integrated manner. It also gave me insight into the role of simulation and modelling in engineering decision-making, which I believe is important in the development and evaluation of future aerospace systems.

Microsoft Copilot, OpenAI ChatGPT, and Anthropic Claude were involved in helping me to interpret the equations and functions, and generate specifications of aircraft to test the code as they are more advanced than my current course. Learning and research was done solely without the help of GAI, and coding was planned and executed by myself with the help of my IEP tutor where required.

Sources:

Factors affecting takeoff distance:

<https://eaglepubs.erau.edu/introductiontoaerospaceflightvehicles/chapter/takeoff-landing-performance/>

CD_0 : European Organisation for the Safety of Air Navigation BADA Manual

<https://www.aircraft.airbus.com/en/newsroom/stories/2024-02-airbus-to-introduce-a330neo-takeoff-performance-enhancements>

https://www.primidi.com/airbus_a330_mrtt/specifications

<https://simpleflying.com/what-to-know-airbus-a330-mrtt/>

https://en.wikipedia.org/wiki/Density_of_air